



Group18

Design and build 2025

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

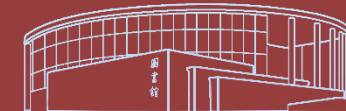
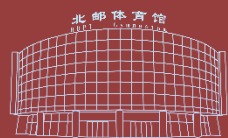


2025/10/20

Team members

- 01** | Jialing Ma: Frontier Point Filtering,
Map Denoising, Host System
Architecture Design and coding
- 02** | Zhenzhan Zhao: Visualization
- 03** | Tianxiao Ma: A* Algorithm
- 04** | Zhichun Tang: Decoding Pose
- 05** | Zhengxiao Zha: SLAM Algorithm

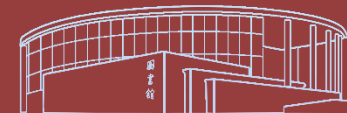
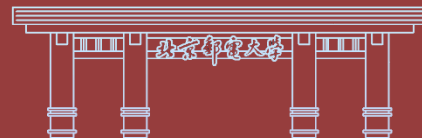
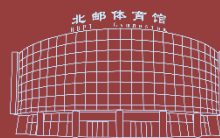
- 06** | Qiran Sun: System Architecture
- 07** | Shangyang Li: Motor Drive
- 08** | Ran Sun: Bluetooth Data Transmission
- 09** | Chuqi Liu: Radar Data Processing
- 10** | Kaixi Xu: MPU Attitude Sensing
- 11** | Jiaqi Yin: System Integration



PART

01

Software Code

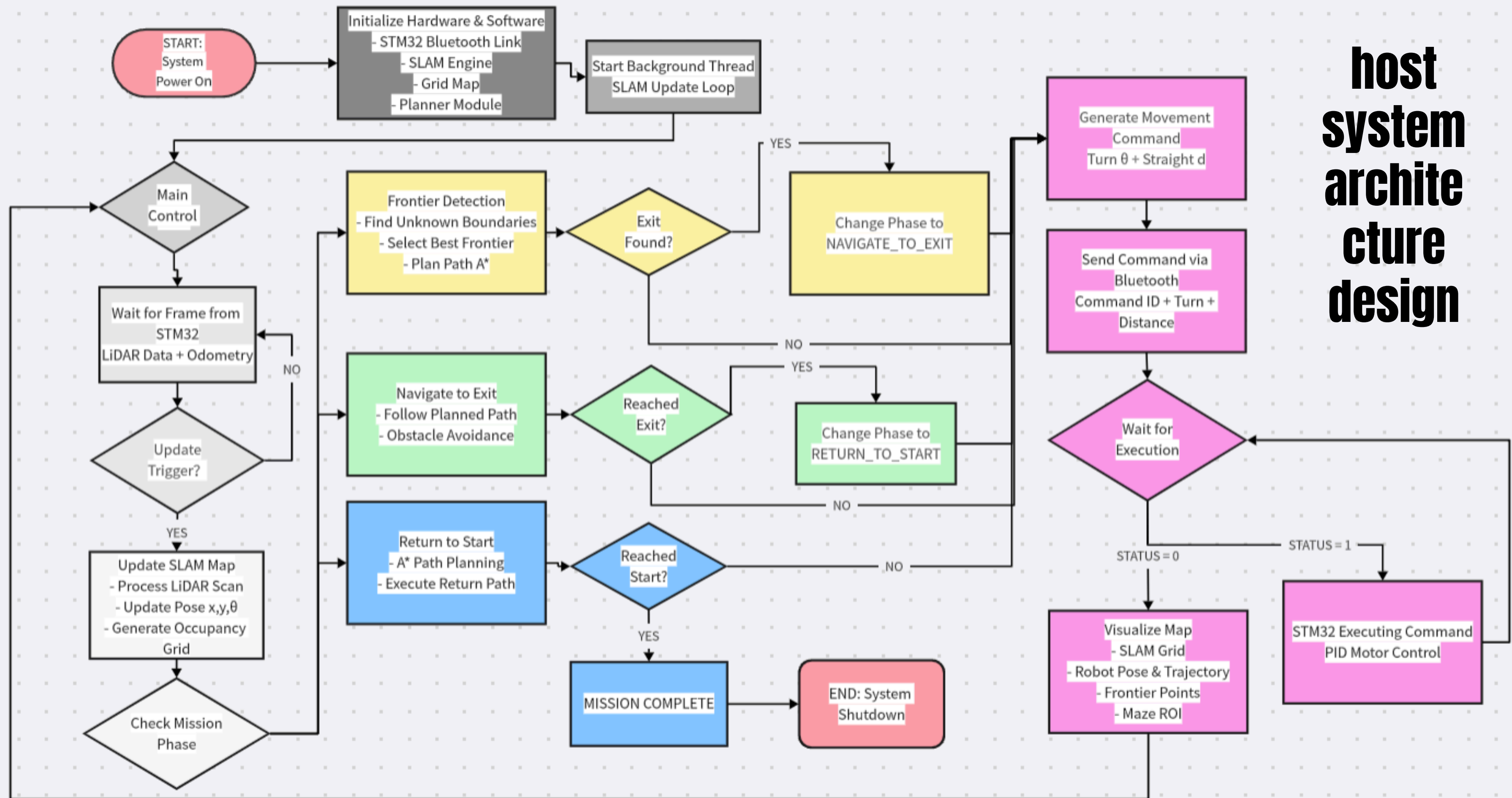




Software part

Project objectives and overall planning

The core objective of this project is to build a fully autonomous intelligent navigation system. This system serves as the "brain" of the car, enabling it to independently complete intelligent exploration, path planning and other tasks in a completely unknown maze environment, relying solely on the real-time map constructed by SLAM. Finally, within the shortest time, it can find and plan a path to the maze exit and successfully drive out of the maze.





Software part



State Machine

We have defined clear task stages, including EXPLORE, GO_TO_EXIT, RETURN_HOME, etc.
This is the "chief commander" of the system, ensuring that the car performs the correct macro tasks at the right time.



Decision & Planning

OptimizedFrontierExplorer :
Selects the best frontier point using a funnel-based algorithm considering connectivity and goal direction.
GlobalPathPlanner: Uses the A* algorithm to compute the shortest pixel-level route to the target.



Safety & Execution

MapRectifier: Identify the effective boundaries of the maze to prevent wasting time outside of it.
Obstacle Inflation: Inflate all obstacles to ensure that all planned paths maintain a safe distance from the walls.
waypoint_to_turn_and_go: Finally, the system decomposes the complex A* path into specific action instructions and hands them over to the underlying hardware for execution.



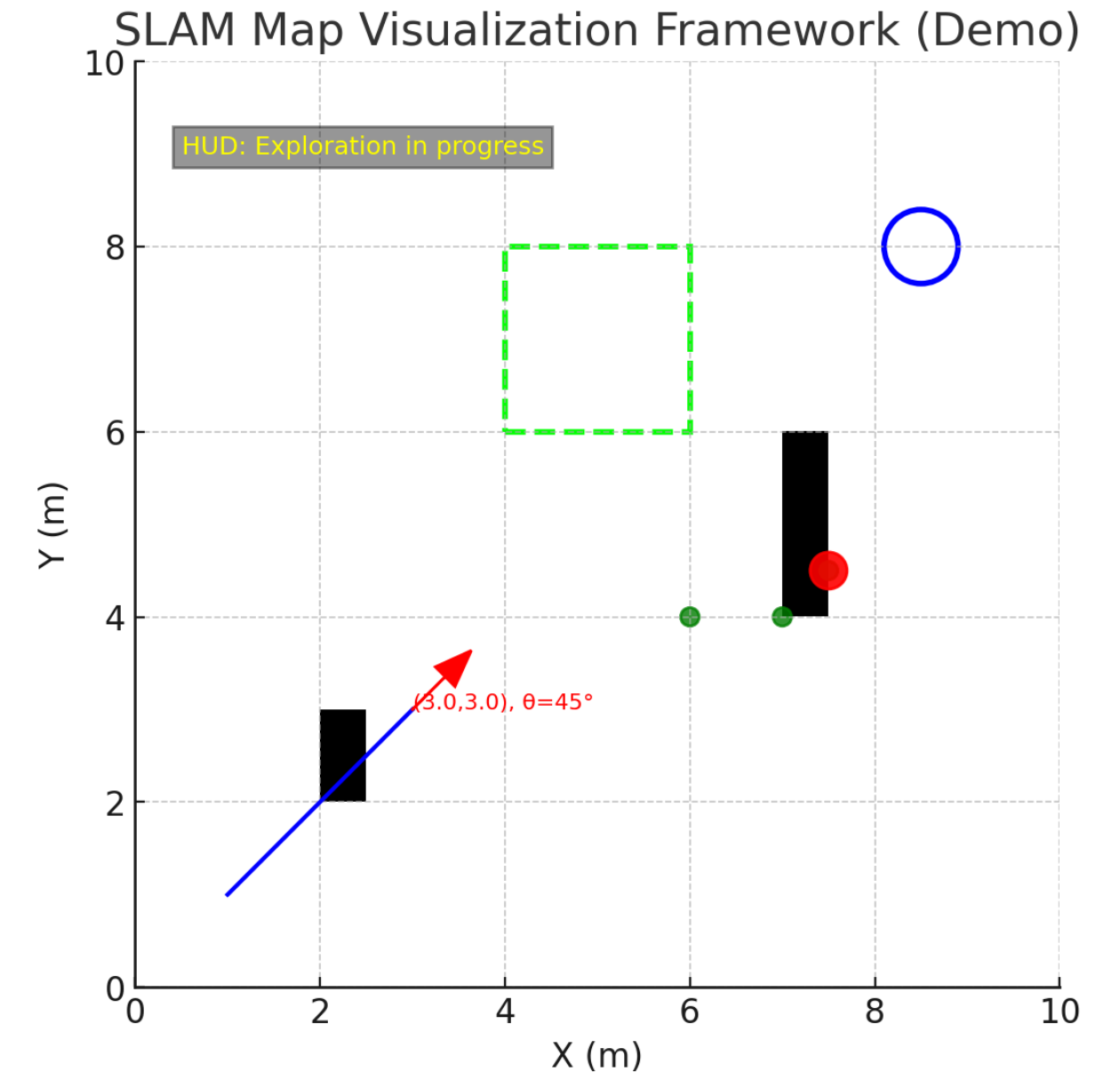
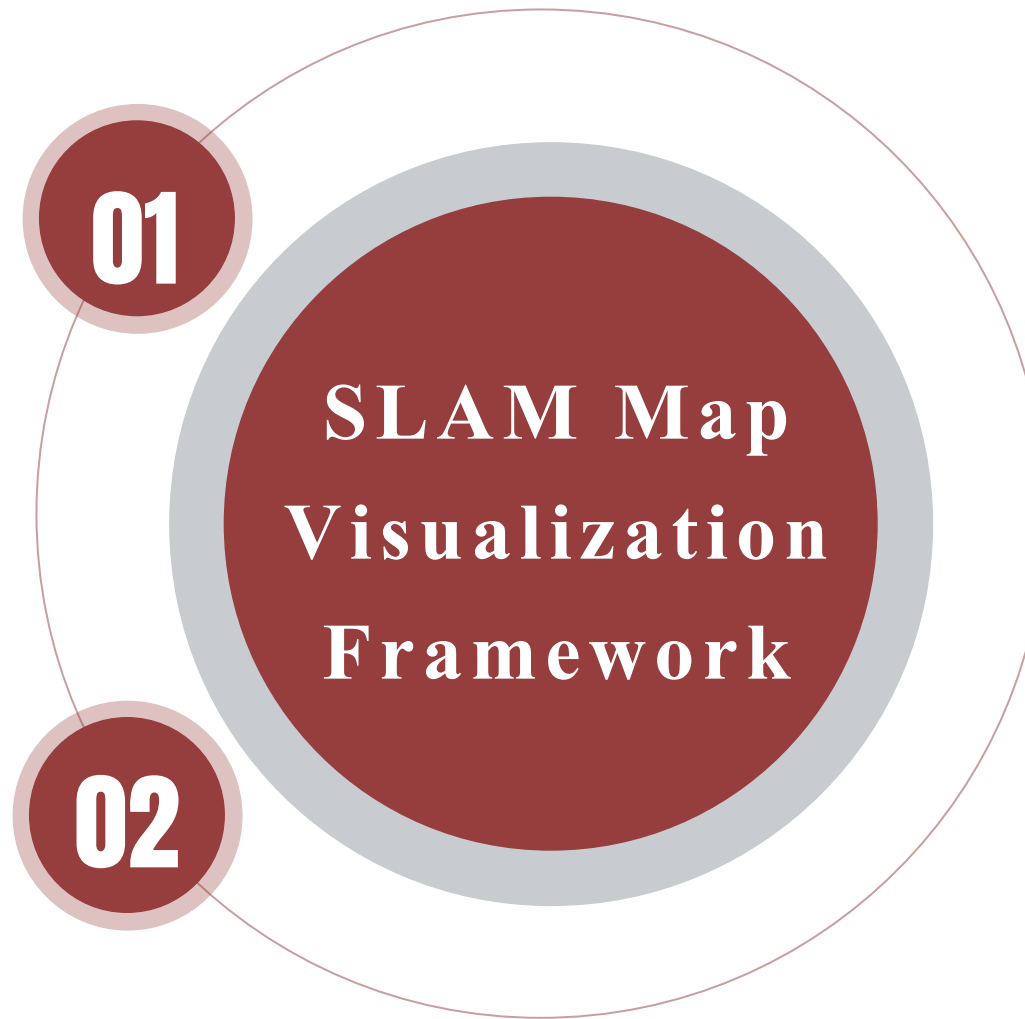
Visualization

Core Objectives

- Display SLAM occupancy grid map dynamically
- Track robot pose and trajectory in real time
- Overlay additional markers (frontier points, exit points, ROI)
- Provide intuitive visualization tools for debugging and experiments

System Structure

- Map Display: Occupancy grid shown as grayscale image (imshow)
- Robot Rendering: Red arrow indicating position and orientation
- Trajectory Tracking: Blue polyline recording historical path
- HUD Text: Extra debugging information (yellow text with black background)
- ROI Rectangle: Green dashed box highlighting region of interest





Visualization

01

Robot & Trajectory

- Red Arrow → Current robot pose and heading (`_draw_car()`)
- Text Label → Real-time state (x, y, θ) (`_draw_car()`)
- Blue Line → Historical trajectory (`_update_traj()`)

02

Frontier & Exit Points

- Frontier Points → Exploration boundaries (green/blue small circles) (`_draw_frontier_points_extra()`)
- Best Frontier → Highlighted red circle (`_draw_frontier_points_extra()`)
- Exit Pose → Blue hollow circle (`_draw_exit_pose()`)

03

ROI Region

Green Dashed Box → Marks region of interest (`display()` with `mpatches.Rectangle`)

04

Dynamic Refresh

- Real-time Update → `plt.pause(0.01)`
- Artist Reuse → Prevents ghosting, improves performance



SLAM Algorithm

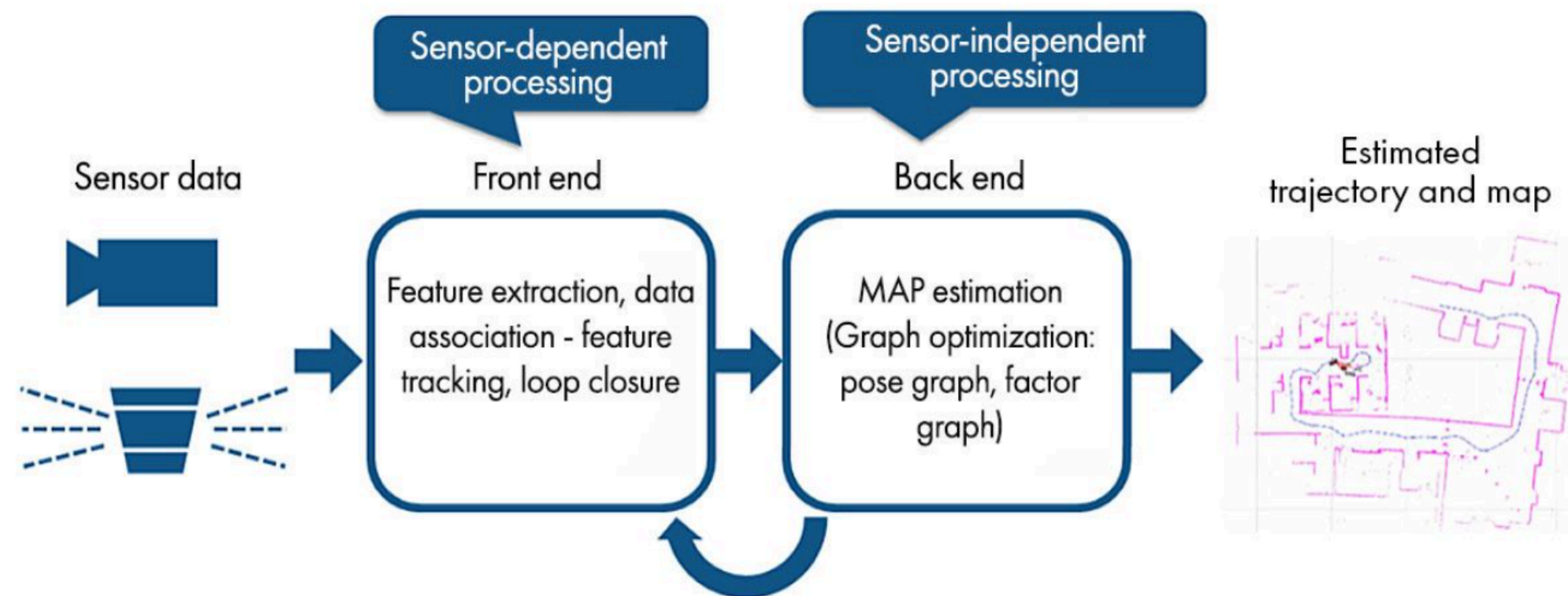
We employ the SLAM (Simultaneous Localization and Mapping) method to enable robots to simultaneously localize themselves and construct maps in unknown environments. With each movement, the robot must not only utilize the existing map to estimate its new position but also update and expand the map based on new sensor observations.

Core Idea

We use the SLAM (Simultaneous Localization and Mapping) algorithm to help robots simultaneously locate themselves and build a map in unknown environments.

"Simultaneous" means localization and mapping depend on each other and happen in real time — one cannot work without the other.

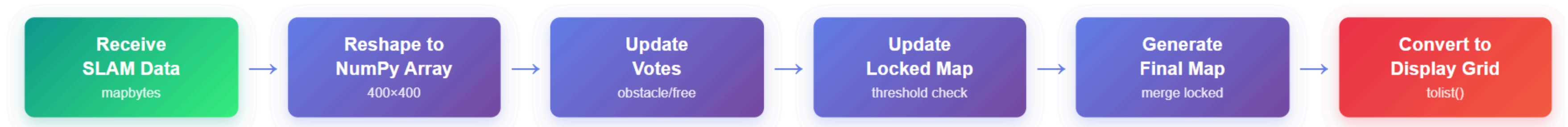
As the robot moves, it must both estimate its new position using the existing map and update the map with new sensor data.





MAP update

Since the system uses BreezySLAM for real-time localization and mapping, the raw occupancy grid produced in each frame often contains considerable noise and temporal instability



To achieve a clean, stable, and geometrically reliable map, a series of post-processing algorithms were designed for noise suppression, temporal fusion, and structure preservation.

Voting-Based Temporal Fusion

Implemented in `_update_votes()`

Each grid cell keeps two counters:

`obstacle_votes[y][x]` and `free_votes[y][x]`.

Every new SLAM frame updates these votes:

Obstacle pixel → add one obstacle vote.

Free pixel → add one free vote.

Unknown → ignored.

Locked cells are excluded from updates.



Locking Mechanism for Stable Cells

Implemented in `_update_locked_map()`

A cell becomes locked (permanently classified) when votes exceed certain thresholds



Final Map Generation

Implemented in

`_generate_final_map()`

Merges locked information with the latest SLAM output:

Locked cells → from `locked_map`

Unlocked cells → from recent `slam_map`

Produces a clean, fused occupancy grid ready for navigation and exploration.



A* Algorithm

We use the A* algorithm (A-Star) to plan the optimal path from the starting point to the target point.

The so-called "optimal" refers to the shortest distance that a path travels under the premise of feasibility.

core idea

Each time you expand a grid, you should not only look at "how far you have gone", but also consider "how far you are from the goal". It finds the current point with the greatest hope of approaching the finish line by integrating two parts of information.

$$f(n)=g(n)+h(n)$$

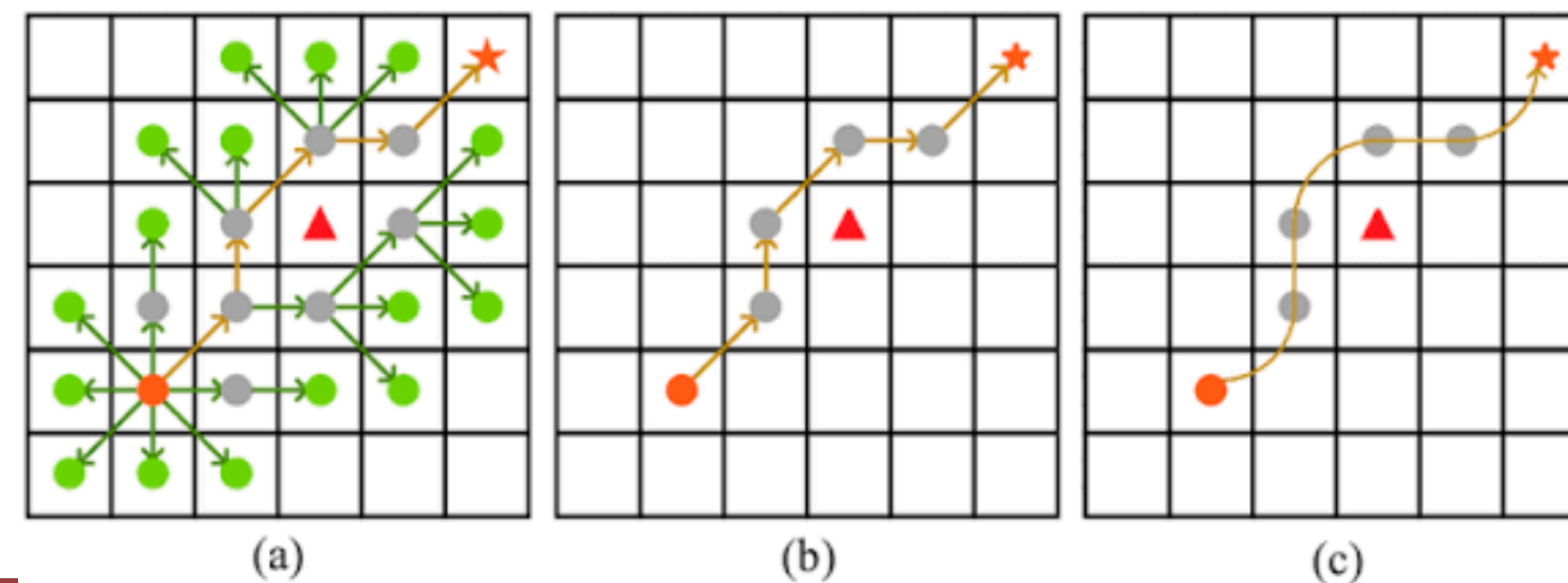
$g(n)$: The actual cost of walking from the starting point to the current node.

$h(n)$: The estimated cost from the current node to the target.

$f(n)$: Total cost = Actual + Predicted. The system prioritizes expanding the node with the smallest f .

core cost function

★ target ● start point ▲ obstacle ↗ parent relationship for other node
● node in closelist ● node in openlist ↘ parent relationship for current node





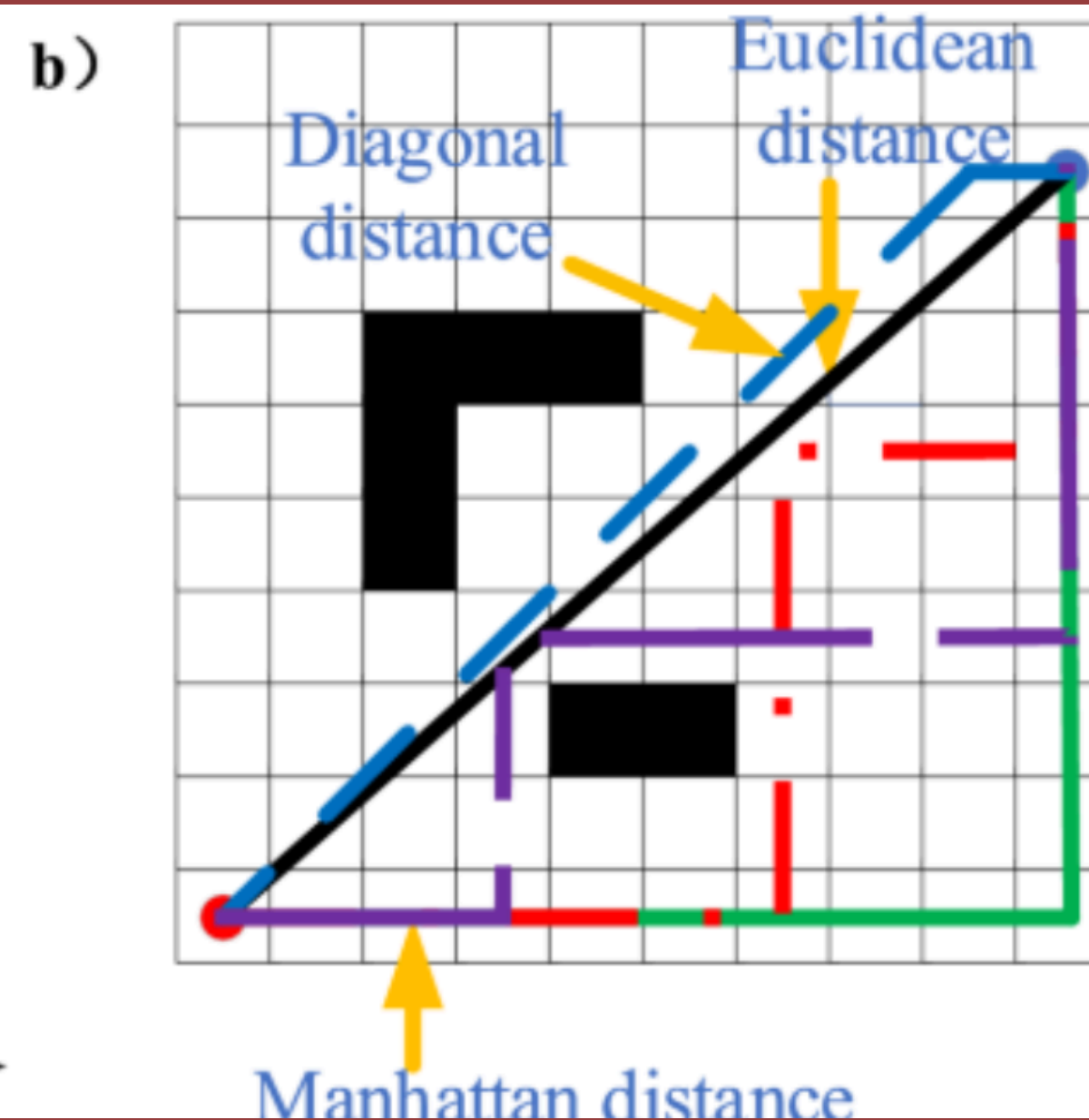
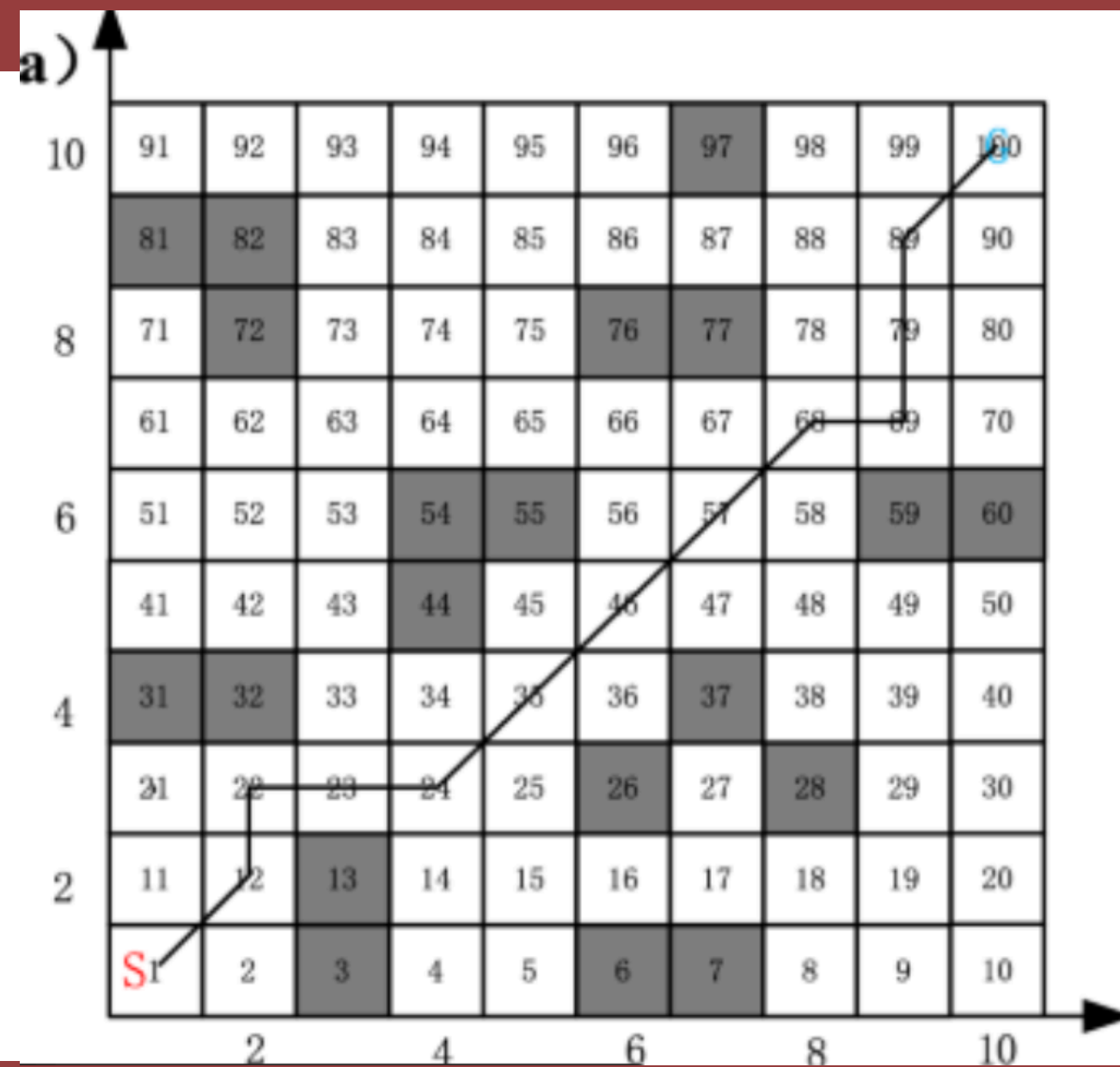
A* Algorithm

Path Simplification

The path output by the A* algorithm is a "zigzag line" composed of pixel grid points. For example, the output path of A* is:

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3)$

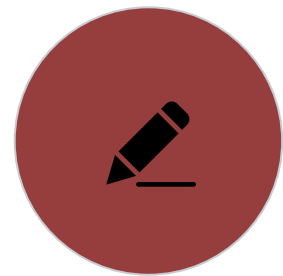
But if we don't simplify it, the robot will treat each of these five grids as an independent action to execute





Optimized Frontier Explorer

by introducing multi-layer filtering and prioritization to select the most informative and reachable frontier points efficiently.



01

filter by distance

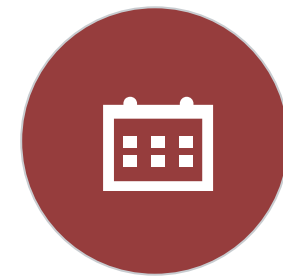
Check whether at least 70% of the area around the goal is free of obstacles, ensuring a safe landing zone.



02

spatial sampling

Reduce the number of frontier candidates in dense areas.

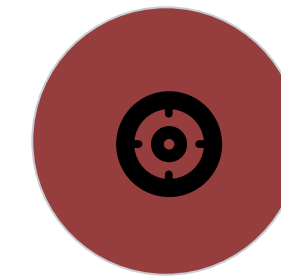


03

filter by connectivity simple

Ensure the candidate frontiers are reachable from the robot's current position.

Perform a BFS (Breadth-First Search) over "robust free space," i.e., regions surrounded by a sufficient amount of free cells ($\geq 70\%$).



04

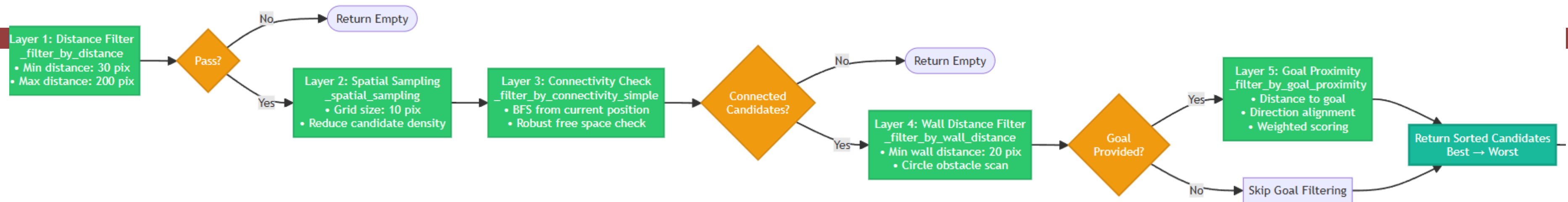
filter by goal proximity

Guide exploration toward the global goal once it becomes partially reachable.

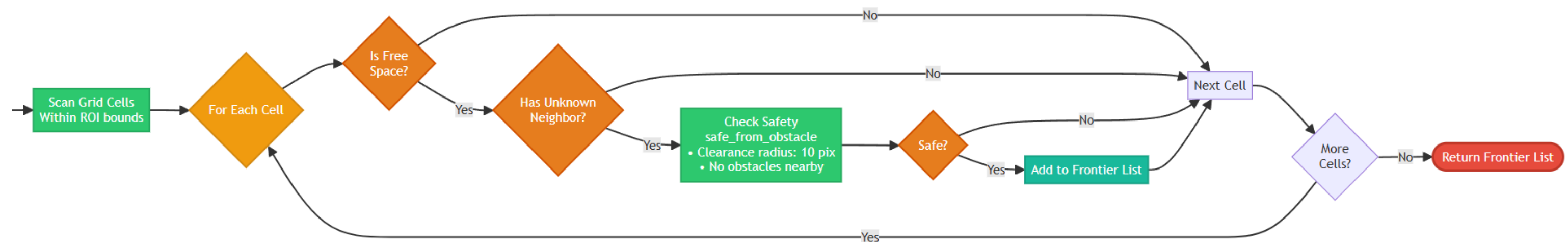


Optimized Frontier Explorer

by introducing multi-layer filtering and prioritization to select the most informative and reachable frontier points efficiently.



Frontier Detection Process





Vehicle Pose Decoding

This module is responsible for collecting real-time environmental and vehicle state data through various onboard sensors. It processes raw data to filter noise and extract key features, ensuring reliable input for subsequent decision-making and control modules.

Bluetooth Protocol (BTLink class)

Defines bidirectional communication frames between PC and vehicle, ensuring reliable command and telemetry exchange.

- `send_cmd(cmd_id, turn_rad, distance_m)`: Sends movement commands to the vehicle.
- `get_latest_frame()`: Retrieves the latest telemetry data frame from the vehicle.

Lidar Data Processing (`build_scan360` function)

- Input: Raw LidarPoint list with quality, angle_q6, distance_q2.
- Process: Decodes, aggregates by angle, interpolates gaps, reorders to CCW.
- Output: A 360-element list of ranges in mm.

Pose Decoding (`ticks_to_motion` function)

- Input: Previous and current encoder ticks (encoder_l, encoder_r, time_us).
- Process: Uses WheelGeom params to compute (dxy_mm, dtheta_deg, dt_s).
- Output: Velocities for SLAM.

Generate SLAM Inputs (`to_slam_inputs` function)

- Input: prev_state, UpFrame, WheelGeom.
- Process: Calls `build_scan360` and `ticks_to_motion`.
- Output: (scan360_mm, (dxy_mm, dtheta_deg, dt_s), curr_state).

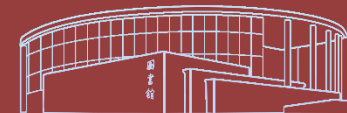
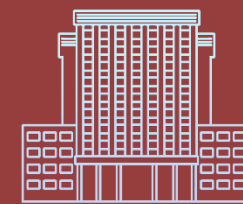
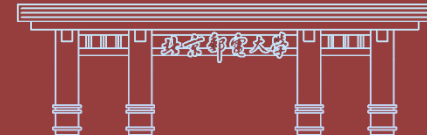
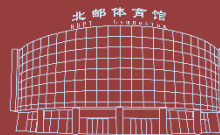
Execute SLAM Update

If all data is valid, pass the outputs to `BreezySLAM.update()`; otherwise, await the next frame.

PART

02

Hardware Introduction





System Architecture Design

Define the high-level system architecture, including hardware and software components.

Specify the interaction between sensors (2D LiDAR, odometry), processing units (e.g., Raspberry Pi, Arduino), and actuators (motors).

Design the communication protocols between components.

Sensor Integration and SLAM Implementation

Sensor Integration

Integrate the 2D LiDAR sensor to capture environmental data (angle and distance measurements).

Implement odometry data processing (x , y , θ) for localization.

Ensure synchronization between LiDAR and odometry data for accurate SLAM.

SLAM Implementation

Develop or integrate a SLAM (Simultaneous Localization and Mapping) algorithm to construct a map of the maze in real-time.

Navigation and Path Planning

Path Planning

Integrate global path planning using the A* algorithm.

Implement local obstacle avoidance using the Dynamic Window Approach (DWA).

Ensure seamless switching between global and local planning for efficient and safe navigation.

Bluetooth Communication

Establish Bluetooth communication (e.g., using Arduino Blue or similar) for sending control commands to the robot.

Enable manual control and high-level command execution (e.g., start, stop, return to start).

Simulation Environment and Real-World Deployment

Simulation Environment

Set up a simulation environment to test navigation and mapping algorithms.

Simulate LiDAR and odometry data to validate the SLAM and navigation stack before real-world deployment.

Real-World Deployment and Testing

Deploy the integrated system on the physical robot.

Test the robot in both structured (9-cell, 16-cell) and random mazes.

Optimize performance for speed, accuracy, and reliability.



Bluetooth Communication Protocol & Data Transmission

This module is responsible for the **bidirectional communication link** between the vehicle and the host computer. It handles **command reception and status feedback**, providing the fundamental communication capability for remote control and data synchronization.

Downlink Protocol

Frame Format:

0xAA 0x55 | LenL LenH | Payload | CrcL CrcH

State Machine Parsing:

The *Bluetooth_Uart_Receive_Handler* runs under HAL UART interrupt and transitions through the states above.

Once complete, data is stored in *Bluetooth_Receive_buff* and the flag *Complete_Bluetooth_Receive_Frame* is set.

Command Format:

cmd_id(2B) | turn_rad(4B) | straight_m(4B)

The decoded result is passed to *Car_FSM_Cmd_Set()* for execution. New commands are accepted only when the car is in the STOP state.

Buffer & CRC Check:

The receive buffer is dynamically allocated and reset. CRC verification is currently a placeholder (if(1)), ready for future implementation.

Uplink Data Transmission

Frame Format

0x55 0xAA | Payload | Crc

Payload Includes

Timestamp

Command ID

Vehicle Status

Encoders of Left & Right Motors

Current Yaw Angle

RPLIDAR_Send_Data

Data Preparation

Before each transmission, data from car state, IMU, motor encoders, and LiDAR are synchronized.

car_state

mpu6500_data

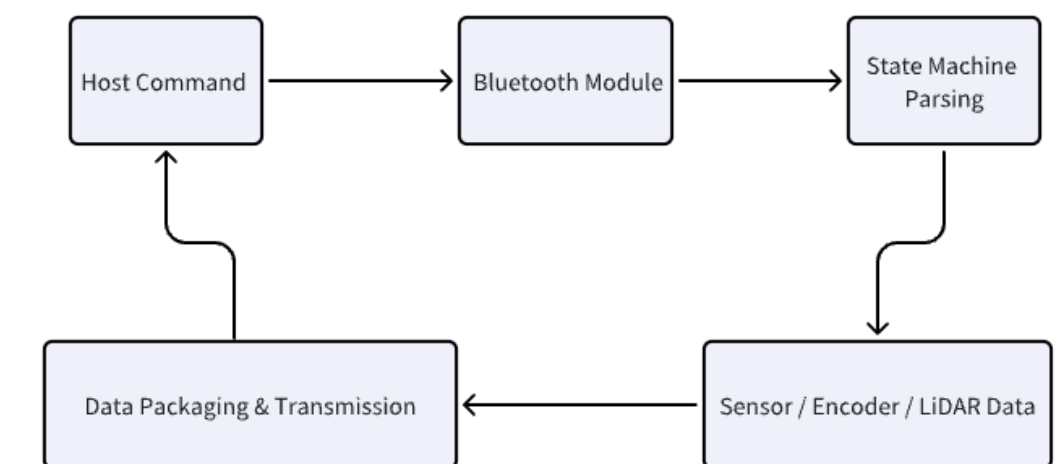
l_motor/r_motor

last_rplidar_data。

The transmitting side uses DMA (*HAL_UART_Transmit_DMA()*) for segmented transmission. Once completed, *HAL_UART_TxCpltCallback()* sets *bluetooth_send_enable* to continue sending.

Communication Workflow

In this system, the Bluetooth module serves as the communication bridge between the host computer and the on-board control core. The host sends command frames to control vehicle actions, while the lower computer parses and executes them. Meanwhile, it packages real-time attitude, encoder readings, and LiDAR data and transmits them back to the host, forming a closed-loop data exchange and feedback mechanism.





Lidar Data Processing and Environmental Perception

The system mainly obtains environmental point cloud data through RPLIDAR, and then completes perception tasks through the following process: LiDAR raw data acquisition, polar coordinate to Cartesian coordinate conversion, noise filtering and effective distance screening, obstacle detection, environmental modeling, and output control signals

LiDAR Data Acquisition

Serial Data Reception

The LiDAR continuously streams scan frames via UART.

Each frame includes angle, distance, and signal strength values.

Frame Parsing

[Header][Angle_L][Angle_H][Distance_L][Distance_H]
[Quality]

Polar to Cartesian Conversion

LiDAR provides polar coordinates (r, θ). The code converts them to Cartesian (x, y):

```
x = distance * cos(angle * PI / 180);  
y = distance * sin(angle * PI / 180);
```

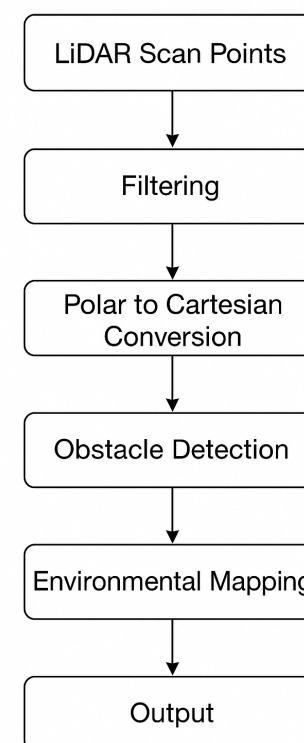
Noise Filtering and Validation

Range threshold

Signal quality filtering

Smoothing filter to remove sharp jumps between consecutive points.

Obstacle Detection and Environmental Modeling



Environmental Perception and Control

The processed LiDAR data are used for:

Obstacle avoidance – detecting obstacles in front or near the vehicle.

Path planning – marking non-traversable areas.

Visualization – transmitting scan results via Bluetooth for real-time display.

Environmental Perception and Control

Before sending, LiDAR data are synchronized with other sensors such as IMU and wheel encoders. The system combines and packages various sensor data in the `bluetooth_send_data()`. This fusion enables more accurate pose estimation and map alignment.



Motor drive & PID control

The motor module adopts a dual-wheel differential drive design. It controls the left and right wheels independently through two separate motors to achieve actions such as moving forward, backward, and turning. The motor speed is controlled by PWM, the position information is provided by the encoder, and the PID controller is used to achieve precise speed and position control.

Multi-level PID control

The multi-level PID adopts an engineering design of "external slow, internal fast, decoupling and hierarchical, limiting and preventing saturation, normalization and anti-jitter", achieving closed-loop control of the differential chassis with stable straight-line operation, smooth steering, and rapid response.

Cascade control: Position loop (outer loop) → Speed loop (inner loop) → PWM output. The angle loop is responsible for heading control, and the speed difference ensures linearity and synchronization of the two wheels.

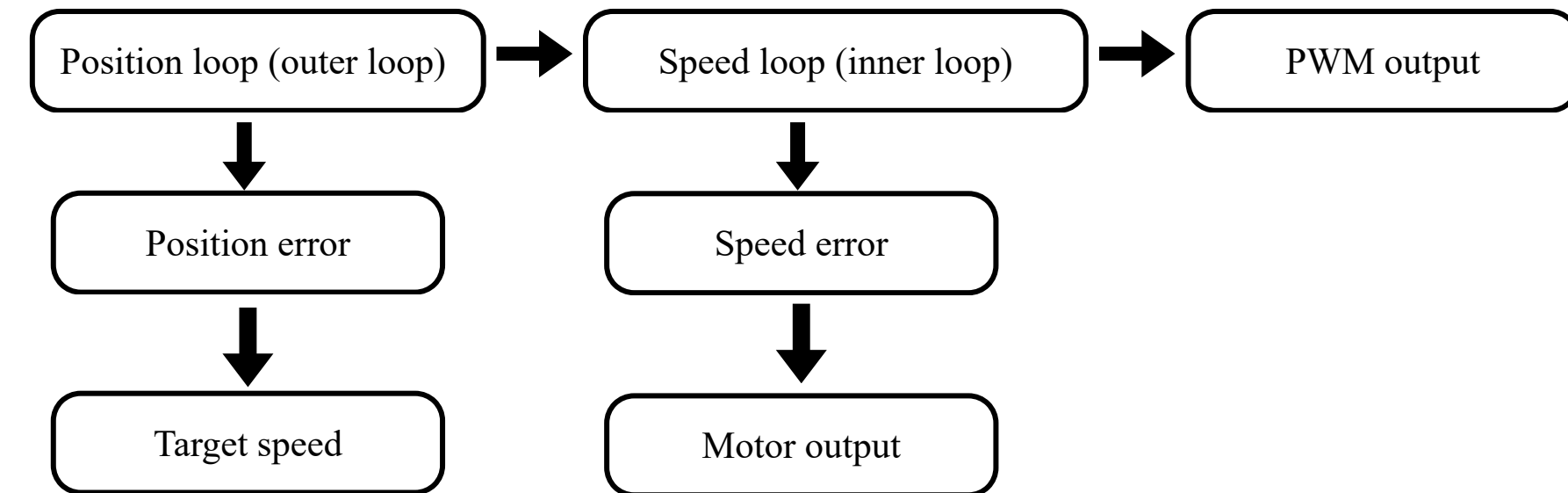
Each loop has clear responsibilities, clear sequence, and parameters are layered, reducing coupling and mutual interference.

Two-wheel differential drive

The differential drive chassis generates angular velocity through the difference in linear speeds of the left and right wheels, enabling straight driving, turning, and spinning on the spot.

Kinematic relationship: $v = (v_L + v_R) / 2$, $\omega = (v_R - v_L) / L$. In terms of control, the target speeds v_L and v_R for the left and right wheels are given, and then the closed loop is converted to PWM.

Each wheel uses two PWM channels. The mutually exclusive output determines the direction, and the duty cycle determines the force.



Real-time feedback

Using a 1ms timer interrupt, the encoder and timestamp are uniformly sampled. The main loop calculates the speed based on the increment and microsecond timing, ensuring the real-time, consistency and anti-jitter capabilities of the feedback.



IMU Attitude Estimation & Orientation Computation

Calculating the real-time attitude of the vehicle using the MPU6500 inertial measurement unit.

By fusing accelerometer and gyroscope data, it provides accurate roll, pitch, and yaw estimation for SLAM localization, ensuring smooth motion control and improved mapping precision.

Sensor Overview

Device Specification:

MPU6500 Six-Axis IMU (3-Axis Accelerometer + 3-Axis Gyroscope)

Communication Interface:

I²C bus connected to STM32 microcontroller

Function Description:

The IMU continuously measures the car's linear acceleration and angular velocity.

Raw sensor data are acquired by the STM32 at a fixed sampling rate and passed to the filtering module for further processing.

This data provides a reference for estimating the vehicle's spatial orientation, compensating for wheel-encoder drift, and stabilizing SLAM localization.

Data Processing & Fusion Algorithm

Acquisition and Filtering:

The STM32 reads raw data via I²C and applies noise suppression through low-pass and complementary filtering.

Accelerometer readings correct long-term drift, while gyroscope integration provides high-frequency motion response.

Attitude Computation:

The fusion algorithm combines both sources to output Euler angles (Roll, Pitch, Yaw) or quaternion representation.

Algorithm Options:

- Complementary Filter: Lightweight software implementation, ideal for real-time embedded systems.
- DMP (Digital Motion Processor): Hardware-accelerated fusion inside MPU6500, offloading STM32 CPU load.

System Integration & Code Implementation

Acquisition and Filtering:

The STM32 reads raw data via I²C and applies noise suppression through low-pass and complementary filtering. Accelerometer readings correct long-term drift, while gyroscope integration provides high-frequency motion response.

Attitude Computation:

The fusion algorithm combines both sources to output Euler angles (Roll, Pitch, Yaw) or quaternion representation.

This ensures high-stability orientation even during vibration or fast rotation.

Result: Real-time orientation with low latency (<10 ms), reducing pose drift and improving mapping consistency.



Low-level hardware driver and system integration

Low-level hardware driver development (including core peripherals, sensors, communication interfaces, etc.), hardware abstraction layer (HAL) design, and system-level initialization and module integration ensure that hardware resources are correctly called and modules work together to provide a stable hardware interface for upper-level applications.

Low-level hardware driver development

GPIO Driver

Function:

Implement the initialization of general input and output pins ,level reading and writing, interrupt configuration, etc.

GPIO is configured by directly manipulating registers, balancing efficiency and flexibility; a standardized interface is encapsulated, so the upper layer does not need to pay attention to the details of the hardware registers.

UART communication driver

Function:

Implement serial port initialization , send and receive data in interrupt/polling mode, and support upper-layer data communication.

It supports interrupt reception (to avoid blocking) and polling transmission (simple scenarios), and realizes continuous data transmission and reception through buffer management to adapt to different communication needs.

Sensor driver

taking I2C interface sensor as an example)

Function: Communicate with sensors via the I2C bus to read data (such as temperature, humidity, acceleration, etc.).

Strictly follow the sensor timing requirements, encapsulate the initialization and data reading interface, and the upper layer can directly call it to obtain sensor data without having to pay attention to the I2C underlying communication details.

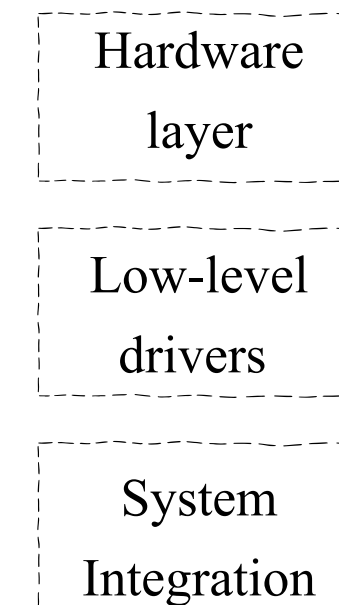
System integration and initialization

System initialization process

Function: Initialize hardware resources (clock, peripherals, interrupts) in order of dependency and establish the system operating environment.

Hardware resource management

Function: Solve problems such as GPIO multiplexing and interrupt priority conflicts to ensure the coordinated operation of multiple peripherals.





北京邮电大学
Beijing University of Posts and Telecommunications

Group18

THANK YOU VERY MUCH FOR
WATCHING



2025/10/20